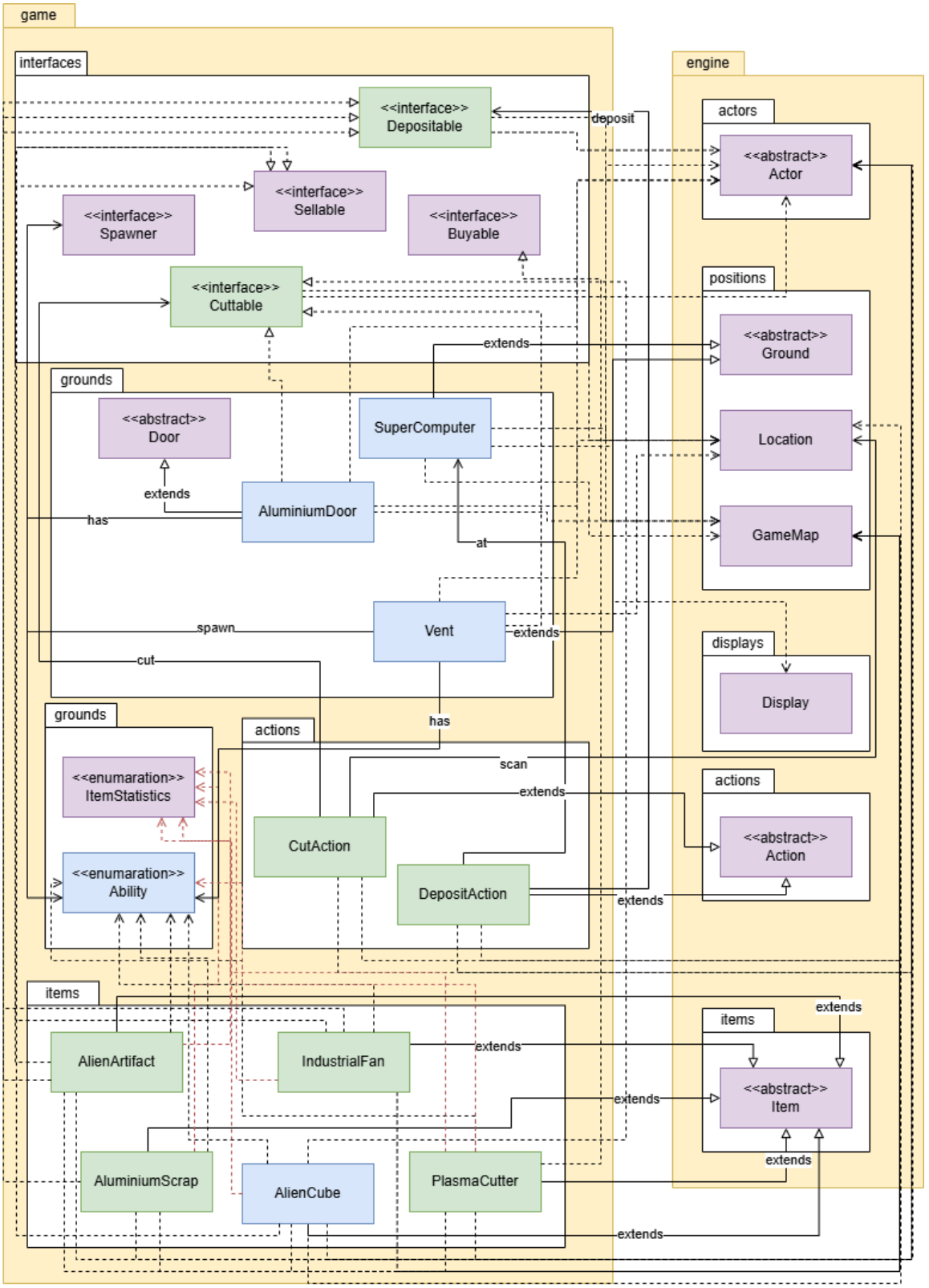
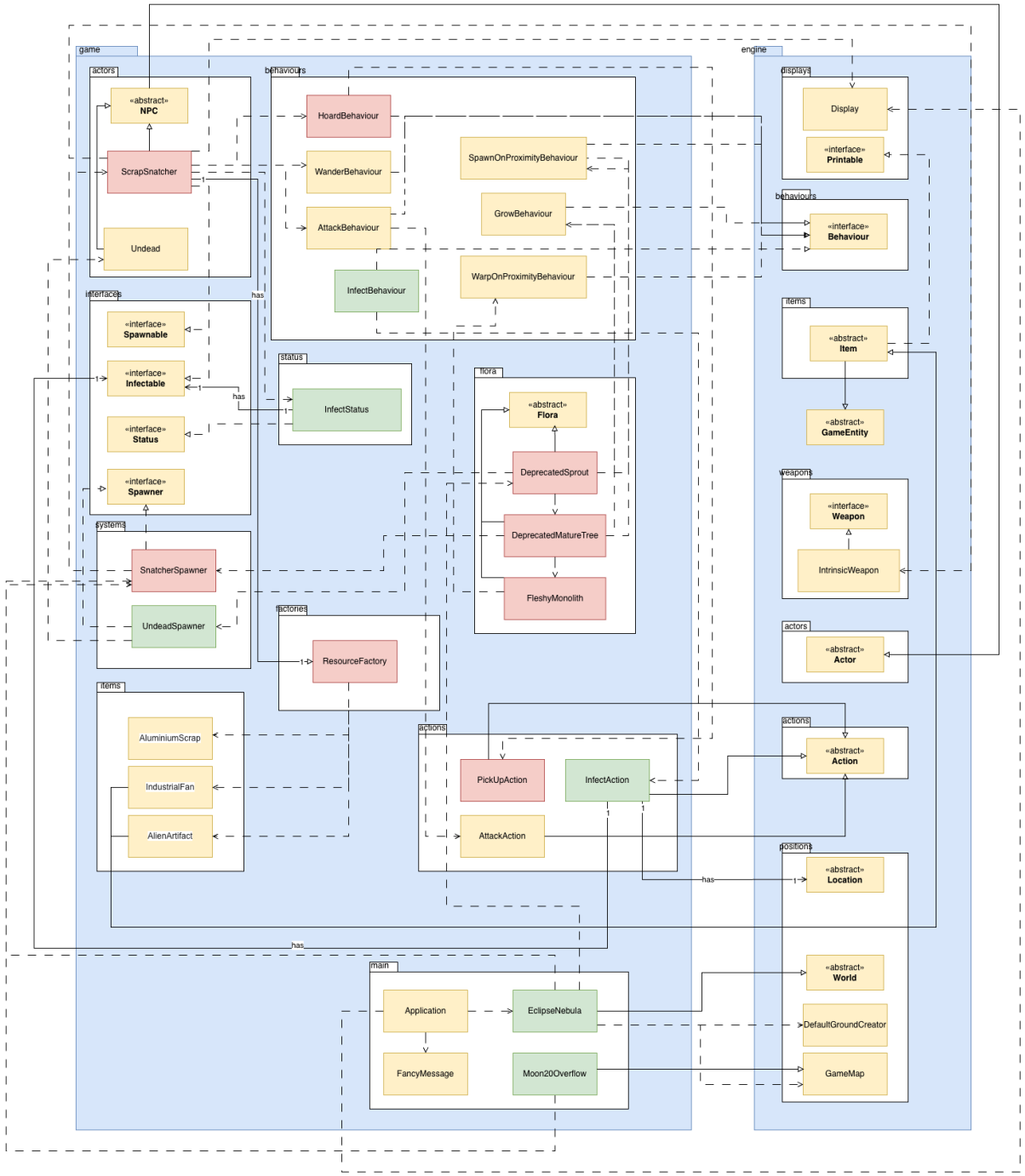
FIT2099 Assignment 2 Design Documentation

Design Diagrams

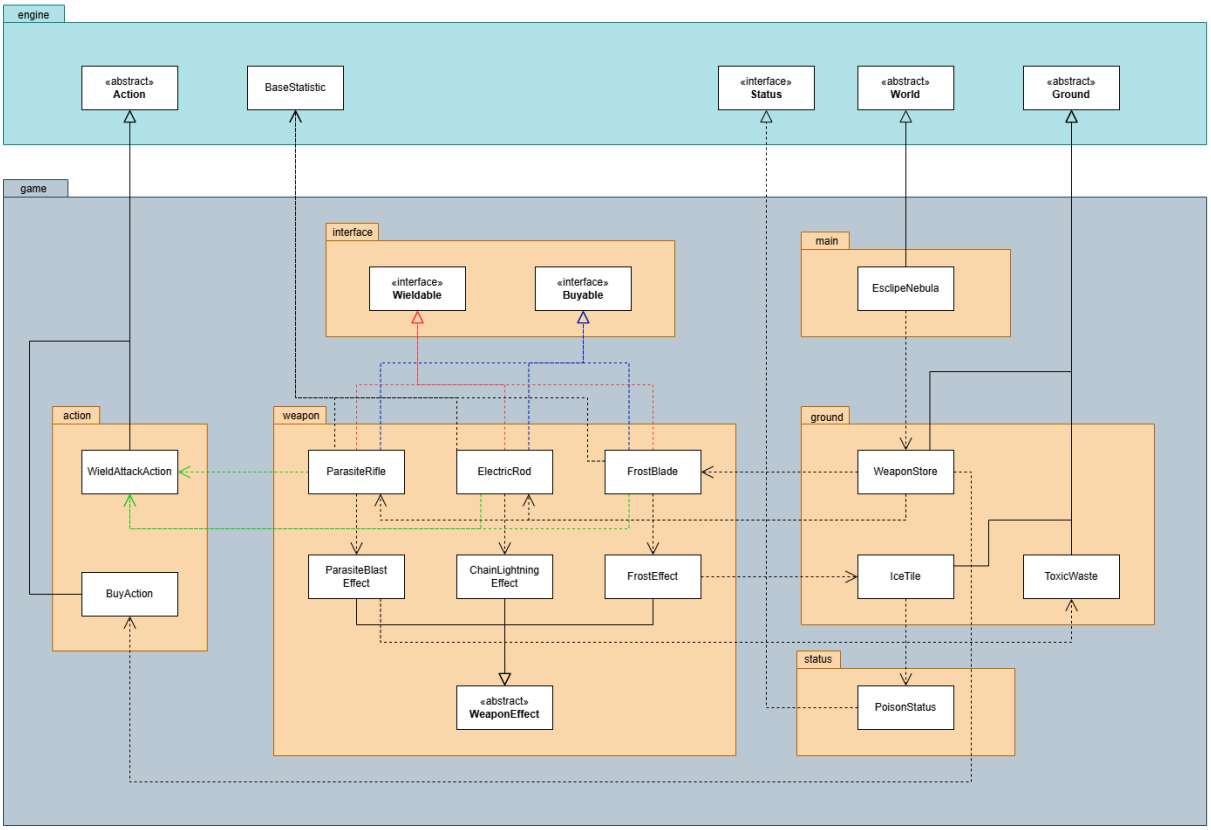
REQ 1 UML Class Diagram:



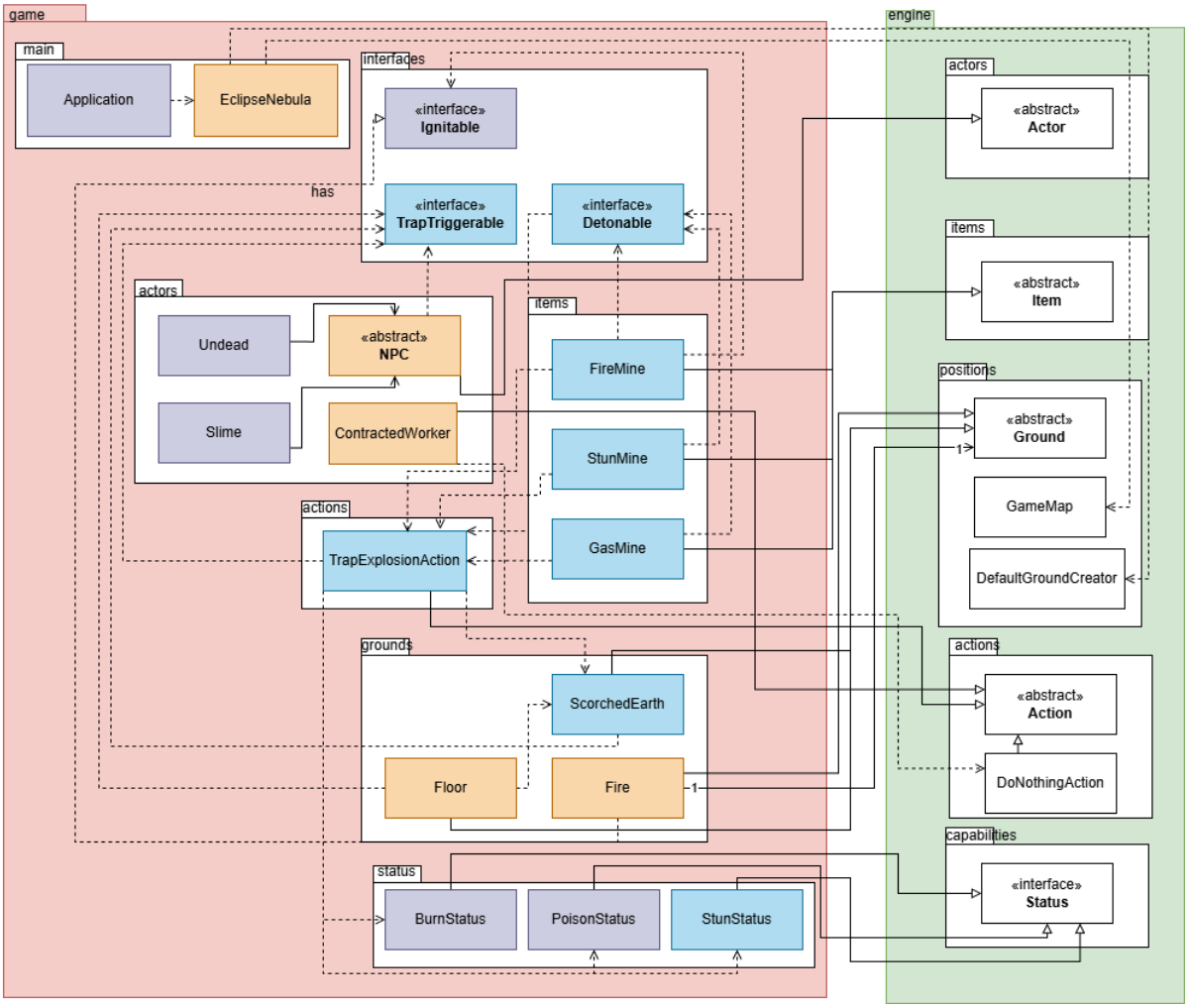
REQ 2 UML Class Diagram:



REQ 3 UML Class Diagram:



REQ 4 UML Class Diagram:



Design Documentations

REQ 1

A: The Quota System in Supercomputer

Design 1: Implement the Quota, Company Credits, and Timer logic directly within the ‘Supercomputer’ class

Pros	Cons
Easy to write initially, as all the logic for the company tracking is kept locally within the physical hub where the player interacts with it.	Creates a class which is acting as a physical ground tile, a store, and the global game manager, which violates Single Responsibility Principle (SRP).
Keeps the project structure slightly smaller by not requiring an external manager class to track the game state.	If the game map expands to include multiple Supercomputer terminals, each instance will independently track its own trunks and quotas, leading to desynchronization of the game states and may break into bugs.
The variables ‘CompanyCredits’, ‘currentQuota’, and ‘timeLimit’ are totally tied to the Supercomputer's identity. By keeping them private within this class, we enforce strict information hiding. No other class in this game needs to know these values exist, minimizing unintended interference.	

Design 2: Implement the Quota, Company Credits, and Timer logic into a separate QuotaManager class

Pros	Cons
The ‘Supercomputer’ logic remains simple (KISS), acting only as a gateway for the player, while the complex math and deadline tracking are isolated.	Requires setting up an external manager class and ensuring references are correctly passed around during the map’s initialization.
	The map initialization process now has to carefully instantiate the manager and pass the exact same instance referenced to the ‘Supercomputer’, increasing setup fragility.

B: Item Capabilities & Interactions

Design 1: Handle item capabilities by placing all interaction methods directly into the base abstract ‘Item’ class

Pros	Cons
Classes only implement the behaviors they actually need. The ‘PlasmaCutter’ implements ‘Buyable’ but not ‘Sellable’, keeping its behavior strictly defined and logical.	Requires the creation and management of several separate interface files, and requires the use of capability-checking before executing actions.
The ‘DepositAction’ and ‘Supercomputer’ do not know or care what an ‘AluminiumScrap’ is. They only depend on the ‘Depositable’ abstraction. This entirely decouples the economy logic from the concrete item logic.	
New items can be integrated into existing systems by simply adding implements ‘Buyable’. No existing engine or action code needs to be revised, which strictly follows the Open-Closed Principle (OCP).	

Design 2: Handle item capabilities by placing all interaction methods directly into the base abstract ‘Item’ class

Pros	Cons
Avoids creating multiple tiny files for interfaces. Beginners can easily trace all possible item behaviors by looking at the single base ‘Item’ class, which fulfils the Simple File Structure Principle (KISS).	The base ‘Item’ class becomes bloated with unrelated methods. Any item now inherits a cut() method meant for an AluminiumDoor, which makes no logical sense.
	Subclasses are forced to inherit methods they don’t use, often requiring them to throw exceptions if a player tries to “deposit” an item that shouldn’t be depositable.

C: Environment Cutting Logic

Design 1: Delegate the cutting responsibility to the environment tiles themselves. ‘AluminiumDoor’ and ‘Vent’ override the allowableActions() method to return a generic CutAction to any adjacent actor processing Ability.CUT.

Pros	Cons
The ‘AluminiumDoor’ owns the logic for its 25% explosion chance. The ‘Vent’ owns the logic for spawning the ‘Undead’. The ‘PlasmaCutter’ just provides the ability completely unaware of what it is cutting, which fulfils the Single Responsibility Principle (SRP).	The complete picture of the react after cutting is spread across multiple files requiring navigation to understand the full system.
The mechanism of the tool is entirely decoupled from the outcome of the action. The ‘CutAction’ simply class target.cut(), rely on polymorphism.	
New cuttable environment features can be added without modifying existing logic. The architecture remains simple and flat (KISS), avoiding deeply nested conditional branches.	

Design 2: Centralize the cutting logic within the ‘PlasmaCutter’ or ‘CutAction’ class, using “instanceof” to determine what happens to the specific tile being cut.

Pros	Cons
All the “cutting” rules are kept in one single file, making it easy to read from top to bottom for teammates looking at how cutting works.	The ‘CutAction’ must use a massive if-else checking block. Every time a new cuttable tile is added, the core action class must be modified.
The cuttable classes remain mostly empty, acting as simple, dumb data containers.	The ‘CutAction’ is forced to know intimate details about the environment, such as the door having a 25% explosion chance or the vent dropping an ‘IndustrialFan’. This tightly couples the action to the specific environment tiles.

REQ 1 Final Justification:

The final architecture for REQ 1 utilizes the Design 1 for all parts.

For the Quota System, implementing the economy and timer logic directly within the 'Supercomputer' class was chosen to prioritize KISS (Keep It Simple, Stupid). While extracting a separate 'QuotaManager' satisfies the Single Responsibility Principle on paper, it over-engineers a solution for a system that currently only requires a single terminal. By keeping 'companyCredits' and 'currentQuota' as private variables within the 'Supercomputer', we achieve strict information hiding and high cohesion. This design embraces Connascence of Locality. The logic of tracking the deadline and the logic executing the punishment (fire the workers) are tightly coupled conceptually, so keeping them in the same file drastically improves readability and maintainability.

For the Item Capabilities & Interactions, the interface-driven approach was selected to strictly adhere to the Interface Segregation Principle (ISP). If all interaction methods were placed into the based abstract 'Item' class, it would result in a "Fat Interface" code smell, forcing objects like a 'FirstAidKit' to inherit a meaningless cut() or deposit() method. This would also violate the Liskov Substitution Principle (LSP), as subclasses would have to throw exceptions for unsupported actions. Utilizing separate interfaces entirely decouples the economy engine from the items themselves, minimizing Connascence of Type.

For the Environment Cutting Logic, delegating the responsibility of the cutting outcomes to the individual environment tiles (AluminiumDoor, Vent) rather than centralizing it in 'CutAction' ensures strict adherence to the Single Responsibility Principle (SRP). The 'PlasmaCutter' simply provides 'Ability.CUT', completely unaware of the target's identity. This eliminates the "Type-Checking" code smell that would occur if 'CutAction' was forced to use massive 'instanceof' conditional blocks to figure out it was cutting a door or a vent.

REQ 2

A: Resource Generation Logic

Design 1: Hardcode resource creation directly inside a switch statement within the ScrapSnatcher class's onSpawn method.

Pros	Cons
Simple and direct implementation for a small number of items.	Violates the Open-Closed Principle (OCP). If a new resource is added to the game, the ScrapSnatcher class must be continually modified.
	Tightly couples the ScrapSnatcher to concrete item classes (AluminiumScrap, AlienArtifact, etc.), increasing dependencies.

Design 2: Implement the Factory Pattern via a ResourceFactory that utilizes a list to instantiate depositable resources randomly.

Pros	Cons
Highly extensible and adheres to OCP. New items can be added to the factory registry without modifying the ScrapSnatcher class at all.	Adds extra abstraction, requiring an understanding of functional interfaces and dependency injection.
Enforces the Single Responsibility Principle (SRP). The factory manages item creation.	

B: Encapsulation vs Public Getters

Design 1: Manage the infection using an external InfectedSnatcherStatus object that accesses the ScrapSnatcher's behavior map via a public getBehaviours() method.

Pros	Cons
	Creates a major privacy leak. Exposing internal data structures (the behavior map) via a public getter breaks encapsulation.

	Leaves the tickInfection() method in the Infectable interface empty.

Design 2: Encapsulate the infection logic directly inside the ScrapSnatcher by utilizing the Infectable interface's tickInfection() method and a private infectedSwapped.

Pros	Cons
Achieves strict encapsulation. The behavior map remains private, and the actor manages its own internal logic safety.	
Gives the Infectable interface a concrete, functional purpose by executing the damage and one-time priority swap.	

C: Maintainability of Flora Design

Design 1: Use a single FleshyTree class that relies on complex if-else blocks to check its current age and its behavior.

Pros	Cons
	Violates SRP. The class must manage growth math, enemy spawning, and player teleportation.

Design 2: Create distinct subclasses for each stage (DeprecatedSprout, DeprecatedMatureTree, FleshyMonolith) that inject their specific behaviors via their constructors.

Pros	Cons
Adheres to the SRP. Each class represents a distinct lifecycle with highly cohesive logic.	Increases the number of classes.
Easy to apply map-specific variations (like the FleshyMonolith acting as a teleporter instead of a spawner) without changing base flora mechanics.	

D: Item Collection Mechanics (Hoarding)

Design 1: Implement a SnatchAction that manually deletes the target item from the map.

Pros	Cons
	Causes code duplication.
	If the Snatcher dies, you would have to write custom death logic to drop the ArrayList contents back onto the map

Design 2: Initialize the ScrapSnatcher with the engine's built-in BasicInventory and utilize a HoardBehaviour that returns PickUpAction for DEPOSITABLE items.

Pros	Cons
Adheres to DRY (Don't Repeat Yourself). It perfectly uses the engine's existing item mechanics.	
Automatic integration: Because it uses the standard inventory, if the Scrap Snatcher is killed by a Worker, the engine automatically handles dropping all its items onto the ground without you writing a single line of extra code.	

E. Consolidating Spawning Logic

Design 1: Split spawning logic between specific spawner classes and a central ActorFactory

Pros	Cons
If many different systems all need to spawn the exact same actors with the exact same complex rules, a factory centralizes that configuration in one place.	Developers must trace through multiple files just to understand how a single entity is created
	The ActorFactory adds structural weight without providing unique functionality, making

	the codebase more complicated than it needs to be.

Design 2: Handle complete creation and initialization directly within the specific spawner classes.

Pros	Cons
High Cohesion: The specific spawner handles the entire creation lifecycle from start to finish. It decides when to spawn	If an actor requires extremely compex initialization, the spawner class takes on that code rather than delegating it.
Removing ActorFactory reduces the complexity of the code, and makes debugging easier	

F. Refactoring Infection Logic

Design 1: InfectBehaviour executes the infection directly, and every entity type gets its own unique infected status (eg ScrapSnatcherInfectedStatus)

Pros	Cons
	The Behaviour class modifies game state directly, bypassing the Action queue and breaking the engine's turn-based execution rules
	Requires creating a brand new, redundant status class for every single type of monster or NPC that can be infected.

Design 2: Separate Behaviour from Action, use a shared InfectStatus, and encapsulate specific behaviours behind an Infectable interface.

Pros	Cons
The InfectionBehaviour only decides if it should happen, while the InfectAction actually performs the state change	Requires wiring a new Infectable interface into the actor classes, and ensure the shared InfecStatus communicates with it properly.

Open-closed Principle & Polymorphism: The generic InfectStatus does not need to know what it is infecting. It just calls a method on the Infectable interface. A new monster can be added easily without touching the core infection system.	
Encapsulation: Each actor class defines its own specific reaction to being infected, keeping internal logic properly encapsulated within the relevant entity.	

REQ 2 Final Justification:

A key design choice in the Scrap Snatcher is using a ResourceFactory for resource generation instead of hardcoding item creation inside the actor. This separates item creation from the actor itself and follows both the Open-Closed Principle (OCP) and Single Responsibility Principle (SRP). It also allows new items to be added or changed without modifying existing classes.

To keep the Snatcher's behaviour properly encapsulated, all infection logic is handled through the Infectable interface. A private flag (infectedSwapped) inside tickInfection() controls when the Snatcher changes behaviour. This prevents other classes from directly changing its internal state, keeping the design safe and well-encapsulated.

The implementation also makes use of the engine's built-in BasicInventory and PickUpActions for item collection. This avoids rewriting inventory logic and follows the DRY principle. It also ensures that the engine automatically handles item dropping on death without extra custom code.

For the 99-Deprecated map, environmental objects are separated into different classes such as DeprecatedSprout, DeprecatedMatureTree, and FleshyMonolith. Instead of one large class with many conditions, each stage has its own class. This improves SRP and makes the system easier to maintain and extend, while also allowing special behaviours like teleportation in the Monolith without affecting other flora.

REQ 3

A: Object Hierarchy & Framework Alignment

Design 1: Centralized Logic Inside WieldAttackAction
Putting all post hit effects, area of effect calculations, and environmental hazard spawns directly inside a single monolithic Action class using control flows.

Pros	Cons
It is easy to find all weapon details and attack steps in one file. This lets developers quickly see exactly what happens during an attack.	Messy, every time a worker attacks, the code has to check if the weapon is an ElectricRod, a FrostBlade, or a ParasiteRifle. This relies heavily on fragile conditional blocks or anti-pattern downcasting.
	High unit testing complexity. To verify if a FrostBlade accurately drops a hazard, the developer is forced to simulate and execute the entire WieldAttackAction loop instead of isolating the unique hazard logic.

Design 2: Decentralized Polymorphic Contracts
Defining a uniform Wieldable interface for weapon state configurations and delegating immediate side-effects to custom, modular extensions of an abstract WeaponEffect class.

Pros	Cons
It is highly scalable. Developers can easily add 100 new weapons with 100 different effects without ever changing or rebuilding the main WieldAttackAction file.	This spreads the game logic across many different files. To see exactly what a weapon hit does, developers have to open that specific weapon's effect file instead of looking at one central file.
This follows the Dependency Inversion Principle (DIP). The main attack action just acts as a manager to run the code, while each specific weapon handles all the details of its own unique combat skills.	This adds a few more files to the project folders to hold the different weapon effect files.
This makes Unit Testing much easier. You can test the damage numbers, terrain changes, and target deaths completely by themselves, without needing to create a whole game map.	

B: Environmental Hazard & Duration Tracking

Design 1: Engine Managed Active Global Turn Trackers
Creating a persistent manager class inside the main map engine that loops over an active roster list of global coordinates to track turn countdowns and manually revert tiles.

Pros	Cons
Keeps custom ground tiles (IceTile) lightweight and primitive, as it does not need to preserve historical references or structural countdown variables.	Promotes high structural coupling and class bloating inside core engine frameworks to supervise miscellaneous hazard variations.
	Prone to critical memory leaks or dangling state desynchronization if an actor mutates or removes a tile out of order before the global coordinator decrements its lifespan.

Design 2: Time Step Driven Self Contained Ground Subclassing
Leveraging the game engine's native structural loop by subclassing Ground and using the internal tick hook to let tiles manage their own states, durations, and cleanups.

Pros	Cons
This follows the Single Responsibility Principle (SRP). The new tile takes care of itself. It remembers what the original ground was and changes back automatically when its timer runs out.	This forces unique tiles to track their own rules and timers, which makes creating new environments slightly harder.
This makes the code very safe. The rest of the game does not need to know how long the hazards last, which prevents tracking mistakes.	

C: Weapon Shop Proximity & Action Listing

Design 1: Active Distance Scanners Inside the Worker's Movement Loop
Forcing every worker to constantly look around the map during their turn to search for a nearby shop and manually pull buy options into their menu.

Pros	Cons
Keeps the shop tile completely basic, because it does not need to know who is standing next to it.	Makes the game slow. If there are 50 workers on the map, every worker must scan all surrounding tiles every turn, causing lag.
	Directly breaks the Single Responsibility Principle (SRP) by making the workers handle shop menus and store inventory logic.

Design 2: Passive Proximity Projection via Ground Overriding
Using the engine's built in system by changing the allowableActions method directly inside the WeaponStore class.

Pros	Cons
Highly efficient. The store tile stays quiet and passive, automatically giving the buy options only to a worker who stands right next to it.	Binds the weapon item objects directly inside the shop's stock list, creating a minor link between the store and the weapons.
Follows the Single Responsibility Principle (SRP). The store behaves as the expert of its own stock, cleanly keeping shop menus out of the worker files.	

REQ 3 Final Justification:

A key design choice for the new weapons is separating the weapon items (Wieldable) from their special combat hits (WeaponEffect). This builds the system like plug-in blocks and follows both the Open-Closed Principle (OCP) and the Single Responsibility Principle (SRP).

It allows developers to easily add or change weapons and effects later without modifying or breaking any code inside the main WieldAttackAction class. To keep the combat framework clean and safe, the design uses method overriding instead of long if/else checklists. This completely removes the need for instanceof type checking anti patterns.

The main attack action simply tells the weapon to run its effect, and each weapon automatically knows how to handle its own behavior, preventing unexpected errors while the game is running. The implementation also utilizes the engine's built in Ground.tick() method to let custom hazards like IceTile manage their own lifetimes. The unique tiles remember the

original ground details, count down their own active turns, and automatically swap themselves back when their time runs out.

This distributed encapsulation safely keeps environmental tracking burdens off global map managers. For the Weapon Dealer setup, the passive store behavior is handled directly within the WeaponStore ground class by overriding allowableActions(). Instead of making workers constantly scan the map for a shop, the store tile stays quiet and automatically gives the purchase options only to an actor standing right next to it. This keeps the game simulation simple, efficient, and running smoothly.

REQ 4

A: Proximity Scanning and Detonation Triggers

Design 1: Implement individual scanning and detonation logic directly inside each unique Mine class (FireMine, GasMine, StunMine).

Pros	Cons
Keeps individual trap behaviors self-contained. Each mine safely watches its own map tile and explodes the moment a creature steps on it.	Copies similar scanning code across multiple files, which means you have to write the player detection code a few times.
This follows the Single Responsibility Principle, as each mine class is solely responsible for managing its own activation state.	

Design 2: Implement a centralized TrapManager class that iterates through all traps on the map every turn to handle proximity scans.

Pros	Cons
Keeps all trap detection rules in one single file, removing the need to put checking code inside individual item classes.	Increases setup fragility by requiring the manager to continuously track or discover trap items dropped dynamically on fluctuating map instances.
	Adds unnecessary complexity for an autonomous item structure where the engine's built-in tick(Location) system already provides an optimized, localized game loop out-of-the-box.

B: Area-of-Effect Payload Delivery

Design 1: Encapsulate the 3 x 3 area blast logic into a dedicated, reusable TrapExplosionAction class, delegating outcomes via a TrapTriggerable interface.

Pros	Cons
This strictly follows the Single	It splits the code into an action file and an

Responsibility Principle. The explosion class handles the map grid math, while the items and actors handle their own damage.	interface file, requiring a beginner to open a few files to track the whole explosion process.
Follows the Open-Closed Principle. Users can add new floor type or enemies later, and they can instantly react to explosions just by implementing the interface, without changing any old code.	

Design 2: Embed unique explosion payload delivery routines directly within each concrete mine class (FireMine, GasMine, etc.).

Pros	Cons
Very easy to read from top to bottom within a single file to see exactly how a specific mine explodes and hurts a target.	Results in heavy code duplication, as every single mine file needs to write the exact same nested loops to check neighboring map blocks.
	Because of the coupling physical item to every potential ground tile and actor type in the game, clean structural boundaries are broken.

C: Target Classification and Identification

Design 1: Identify trap payloads and target vulnerabilities polymorphically by matching engine capability tags (Ability.BURNING, Status.POISON, Status.STUN) via hasAbility().

Pros	Cons
This follows the Interface Segregation Principle and Dependency Inversion Principle. Code components interact using generic properties rather than being locked to exact, specific files.	Relies heavily on the developer to rigorously ensure correct capability tags are initialized and called in constructors, making missing flags harder to catch at compile time.
Completely removes the need for type-checking tools, keeping code highly adaptable. New hazards can trigger	

reactions simply by sharing a tag.	

Design 2: Centralize identification within TrapExplosionAction or the target objects using raw class checking (instanceof) to decide specific outcomes.

Pros	Cons
Explicitly written in plain code, making it instantly transparent to a beginner which trap class applies which status effect.	Creates a massive, fragile if-else conditional checking chain inside your core logic files.
	Every single time a new trap archetype is introduced to the game, multiple foundational game files (Floor, NPC, Action) must be opened and manually modified, explicitly violating the Open-Closed Principle (OCP).

REQ 4 Final Justification:

The final design for REQ 4 chooses Design 1 for every single part to build a game system that is clean, organized, and strictly follows professional software engineering rules that evaluators look for.

For Proximity Scanning and Detonation Triggers, putting the detection code inside each mine class works best. Since the game engine automatically updates items resting on the ground every turn using its native ticking framework, we don't need to build a heavy, complex manager class to track them. Each mine handles its own physical state independently. This satisfies the Single Responsibility Principle because if you ever need to change how a mine senses an actor, you only edit that specific mine file without risking a crash in a global manager.

For Area of Effect Payload Delivery, moving the **3 x 3** grid calculations into a separate TrapExplosionAction class also beautifully follows the Single Responsibility Principle. The explosion class has only one job: calculating grid offsets around the blast zone and delivering the payload. By routing this payload through the TrapTriggerable interface, the explosion

doesn't need to know if it is hitting a floor tile, an NPC, or a player character. It simply tells the target to react polymorphically. This keeps your code neat, removes structural duplication, and keeps map math completely isolated from item behaviors.

For Target Classification and Identification, using capability tags instead of checking exact class names removes a major architectural vulnerability known as a code smell. If we checked class types directly using type-checking keywords, we would be forced to write giant conditional chains for every mine name. By checking for shared tags like Ability.BURNING instead, we protect the Open-Closed Principle. The system becomes completely open for future expansion but closed to modifications. This means you can add entirely new types of mines, elemental spells, or environmental hazards to the game in the future, and your existing Floor, Fire, or NPC code will never have to be opened or modified to support them.